MULTI-AGENT AI FOR GAME DEVELOPMENT

SLASH ROLLER: ARENA

GAME DESIGN DOCUMENT — FIRST DRAFT

ASSIGNMENT	#01 — First Draft of GDD
AUTHOR	Juan Diego Lugo
DATE	22 July 2026
ENGINE	Unreal Engine 5.8 — native C++ / GAS

1. Executive Summary

1.1 Concept

Slash Roller: Arena is a multiplayer melee-combat deathmatch game with souls-weight fighting. The player is a duelist dropped into one compact arena with up to three other fighters — human or AI — armed with committed light and heavy strikes, a block, a timed parry, a stamina-costed dodge, and one equipped magic ability. Every exchange is a deliberate spend of stamina and animation commitment; every kill is earned in a readable, punishable duel — fought at deathmatch pace, against multiple opponents, on a clock.

1.2 Win and Loss Conditions

- **Win:** score the **most kills when the 8-minute match timer expires.**
- **Tiebreak (FFA):** fewest deaths; if still tied, **sudden death** — no respawns, first kill wins.
- **Team Deathmatch:** the higher team kill total at the timer wins; team sudden death on a tie.
- **Loss:** any other scoreline when the timer ends.

1.3 Modes

Mode	Fighters	Description
Deathmatch (FFA)	2–4	Every fighter for themselves; individual kill count
Team Deathmatch	2v2	Shared team kill pool; respawn away from teammate's fight
Solo / PvE	1 + bots	Same modes vs. bots — bots also fill any empty slot in online matches

PvP and PvE are the same game: bots occupy any unfilled slot, so a match can always start and the game is never dead on arrival.

1.4 Platform, Format, and Production Reality

- **Engine:** Unreal Engine 5.8, **pure native C++**, no Blueprint runtime logic; combat built entirely on the Gameplay Ability System (GAS).
- **Networking:** server-authoritative with client prediction, played over a **Steam listen server** (host + invite). No dedicated servers at this scope.
- **Audio:** engine-native **MetaSounds**, triggered through GameplayCues.
- **Art:** environment and character assets from the Unreal marketplace — art is bought; **all gameplay code is written in-house.**
- **Team & timeline:** one principal engineer / technical director plus a multi-agent AI crew, **5 weeks**, building on an already-shipped combat foundation (GAS core, soft-lock melee, input buffer, Steam sessions).
- **Exit criterion:** a playable **Steam demo build**.

1.5 The Meta-Goal

The development method is part of the deliverable: prove that **one senior engineer directing a multi-agent AI crew can ship a small, professionally complete multiplayer game in weeks** — with the crew producing the arena, the bots, the balance passes, and the QA sweeps as reviewable data, and the human owning architecture and final approval.

1.6 Elevator Pitch

Dark Souls combat weight at *Quake* deathmatch pace, in one arena — built by one engineer and an AI crew in five weeks.

2. Game Mechanics (Player-Facing Actions and Loop)

2.1 The Match Loop

1. **Host or join.** Host a listen server and invite via Steam, or start solo — bots fill every empty slot. Pick a loadout: weapon archetype plus one magic slot.

2. **Fight.** Hunt opponents through the arena. Every takedown scores a kill on the board; the leaderboard and kill feed update live.

3. **Die and respawn.** On death: a 5-second death cam on your killer, loadout swap available while dead, then respawn at the spawn point farthest from active combat.

4. **Timer expires.** Scoreboard, winner banner, one personal coach line built from your actual match data, and a rematch prompt: *"Run it back?"*

2.2 The Combat Verbs

Every verb is a GAS ability with an explicit cost, speed, and counter. All numbers are first-draft tuning targets (full table in Appendix A).

Verb	Speed	Damage	Stamina	Countered by
Light attack	Fast (0.5 s)	12	15	Block (fully negated)
Heavy attack	Slow (1.1 s windup)	28	30	Parry, dodge, outspacing
Block (hold)	—	negates lights	10/hit held	Heavy (guard-break stagger)
Parry (timed block)	0.25 s window	—	12	Nothing — but whiffing it is a free hit
Dodge	0.4 s i-frames	—	20	Prediction (recovery frames)
Magic slot	By ability	18–22	—	12 s cooldown; dodgeable

The triangle: lights beat heavy windups; heavies break block; block (and parry) beats lights. **Parry** is the skill apex: a successful parry staggers the attacker for 1.5 seconds — a guaranteed punish window.

Dodge i-frames through anything but costs the most stamina and has recovery frames a patient opponent can punish.

2.3 Stamina — the Souls Governor

Attacks, dodges, and blocked hits all drain a regenerating stamina pool (100 points; regen 20/s after a 1-second delay from the last spend). At zero stamina the fighter is **winded**: no attacks, no dodges, movement slowed 30%, a visible gasp animation — until stamina recovers to 30%.

Stamina is what makes the combat souls-like rather than a masher: it forces pacing, turns aggression into a spend decision, and makes the winded state the punishment for greed. Every fighter reads every other fighter's winded state (the gasp is visible and audible) — the number itself is private, the *state* is public.

2.4 Magic — the Momentum Swing

Each loadout equips exactly **one** magic ability on a 12-second cooldown: the ranged answer in a melee game, deliberately scarce. First-draft slate:

- **Firebolt** — projectile, 22 damage, travels fast, dodgeable on reaction at range.
- **Shockwave** — AoE burst around the caster, 18 damage, radius 4 m — the anti-gank button, spent on cooldown.

The magic slot may be swapped **only while dead** — a deliberate safe point that doubles as a comeback lever.

2.5 Kill Scoring, Respawns, and the Clock

- A kill credits the last damaging instigator; deaths with no instigator in the previous 5 seconds count −1 (self).
- Respawn is 5 seconds, at the spawn point scored farthest from living fighters — no spawn-camping loop.
- The 8-minute clock is the pressure system: a lead invites hunting the leader; a deficit forces aggression. The compact leaderboard is always on screen.

2.6 The Named Design Risk — Souls Combat in FFA

Committed animations plus free-for-all means a fighter mid-heavy can be hit from behind. This is handled by design, not denial:

- **Small fighter counts** (2–4) in a **single compact, readable arena**;
- The existing **soft-lock / score-based target assist** makes target switching a camera flick, not a menu;
- **Spawn scoring** biases re-entry away from active fights;
- **Shockwave** exists as an equippable anti-gank answer;
- The **winded state is short** (~2.5 s at full regen) so no one is helpless for long.

The Week-1 playtest gate tests exactly this: if third-party ganks feel cheap, the pre-declared tuning levers are arena size, fighter count, and stamina regen — not new systems.

2.7 Why It's a Game and Not a Brawl

Mashing drains stamina into the winded state and dies to the parry punish. Turtling loses to heavies, to the clock, and to a third fighter. Kills require winning committed exchanges under a timer, against opponents who can read patterns. The triangle plus stamina management is the entire skill ceiling — deep enough to practice, small enough to ship in five weeks.

2.8 What the Player Sees (HUD)

- **Bottom-center:** health and stamina bars.
- **Bottom-right:** magic-slot icon with cooldown sweep (driven directly by the ability system).
- **Top-center:** match timer + compact kill leaderboard.
- **Top-right:** kill feed — factual lines instantly, color commentary when available (§3.2).
- **On death:** killer death-cam, respawn countdown, loadout swap buttons.
- **Match end:** K/D scoreboard, winner banner, one coach line per player referencing their real telemetry, rematch prompt.

3. AI Architecture (What Each Agent Does, Through Its Effect on Gameplay)

The architecture has two layers with a hard boundary: a **dev-time crew** of four agents that builds game content through the Claude + Unreal Engine MCP pipeline, and **one runtime agent** that is never load-bearing. In-match opponents (bots) are deterministic C++ state machines — an LLM never steers a fighter, and nothing a model outputs can change a fight in progress.

3.1 The Dev-Time Crew (Claude + Unreal MCP, in-editor)

Arena Architect Agent

- **Role:** blocks out the arena in-editor via the UE MCP — geometry, spawn points, sightlines, cover — from a one-paragraph brief, then iterates from screenshots.
- **Gameplay effect:** the space the player fights in — its sightlines, chokes, and spawn placement — is agent-designed and human-approved.
- **Output:** editor edits + `arena_manifest.json` (bounds, spawn list with scoring hints, landmark names).

Bot Trainer Agent

- **Role:** generates and tunes bot combat parameters per difficulty tier against the stamina/commitment economy.
- **Gameplay effect:** the three bot tiers the player fights — from a masher that winds itself to a reserve-holding parry-punisher — are tuned rows the agent proposes and the human approves.
- **Output:** `DT_BotTuning` DataTable rows — aggression, parry chance, reaction ms, stamina discipline, target-switch bias (Appendix B).

Combat QA Agent

- **Role:** runs automated bot-vs-bot matches nightly, reads logs and screenshots, and files reproducible defect reports.
- **Gameplay effect:** the fairness the player takes for granted — no infinite stagger loops, no spawn kills, no stamina underflow — is regression-tested every night without human hours.
- **Output:** `qa_report.json` — defect, repro steps, severity, build id.

Balance Analyst Agent

- **Role:** reads accumulated match telemetry and proposes tuning diffs when any loadout or verb exceeds win-rate bounds (55% triggers review).
- **Gameplay effect:** no dominant strategy survives a week; the meta the player experiences is actively gardened.
- **Output:** DataTable diff + one-paragraph rationale, human-reviewed before merge.

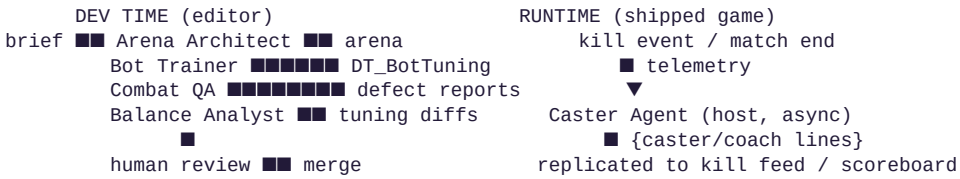
Crew rule: every output is data (JSON / DataTables) that a human reviews before merge. Agents never commit directly. This crew is the capstone's development thesis made demonstrable.

3.2 The Runtime Agent

Caster Agent — the only model call in the shipped game.

- **Role:** generates color-commentary for notable kill-feed events (streaks ≥ 3, parry kills, sudden-death winners) during the match, and one telemetry-grounded coach line per human player at match end.
- **Gameplay effect:** the arena feels broadcast — streaks get called, humiliating parries get narrated — and every player leaves with one specific habit to fix: *"You were parried 6 times — stop opening with the same heavy."*
- **Output format:** per event `{caster_line (≤ 18 words) | null}`; at match end, per player `{coach_line (≤ 30 words, references ≥ 1 telemetry stat)}`.
- **Never load-bearing:** calls are host-side, asynchronous, batched (≥ 10 s apart, ≤ 12 per match + ≤ 4 coach calls). The factual kill-feed line ("A killed B") always renders instantly and locally; a Caster line, if and when it arrives, appends color. Timeout (> 3 s), API error, or cap → canned lines from a shipped DataTable. Players without connectivity lose flavor, never function.

3.3 Interaction Diagram



3.4 Determinism and Trust Boundaries

- **Bots are pure functions** of (tuning row, match seed, observed events): reaction delays are quantized and seeded once at match start. Reproducible for QA; fair for players.
- **The Caster cannot touch the simulation.** Its output is strings. There is no path from a model reply to damage, movement, spawns, or bot behavior mid-match — the earlier design's between-rounds bot-tuning hook was deleted because deathmatch has no safe pause point.

- **The API key never leaves the host.** Clients receive replicated strings only.

4. Technical Strategy (Agent Roles, Token Budget, API Constraints)

4.1 Stack

Layer	Choice	One-line why
Engine	UE 5.8, pure native C++	House standard; no Blueprint runtime logic
Combat	GAS: PlayerState-owned ASC, input buffer, prediction keys	Shipped and stable — reused wholesale
Targeting	Soft-lock (`SoftLockTarget`, replicated) + score-based assist + motion warping	Shipped — the melee feel
Animation	Motion Matching (AL Framework, custom AnimGraph nodes)	Shipped
Netcode	Server-authoritative + client prediction, Steam listen server	Right-sized; dedicated servers deferred behind an interface
Sessions	`OSSessionsSubsystem` (Steam OSS)	Shipped
Audio	MetaSounds via GameplayCues	Engine-native; one less external dependency
UI	CommonUI activatable stack, event-driven GAS bindings	Production pattern; zero polling
New code	Match/respawn flow, stamina + winded, bot state machines, Caster HTTP client	The only new gameplay surface

4.2 Model Assignment and Token Budget

Dev-time crew runs on Claude Code (Sonnet-class) under the project's existing 20-skill UE5 library and rule files — a development cost, not a game cost.

Runtime budget (the shipped game, per match):

Call	Model	Calls / match	Tokens in	Tokens out	Match total
Caster (kill events, batched)	Claude Haiku	≤ 12	600	60	≈ 7,900
Coach (match end, per human)	Claude Haiku	≤ 4	900	80	≈ 3,900

Call	Model	Calls / match	Tokens in	Tokens out	Match total
Total					**≈ 11,800 tokens** **≈ $0.01**

Hard caps enforced server-side (12 + 4 calls). On timeout, error, or cap: canned lines. The token budget is not a risk item at this design — the LLM was deliberately priced out of the hot path.

4.3 API and Engineering Constraints (named, with the decision each caused)

1. **Constraint: multiplayer determinism.** A predicted, server-authoritative action game cannot wait on, or be steered by, a network call to a language model. *Therefore:* the Caster only decorates the kill feed asynchronously and coaches post-match; bots are deterministic C++; canned fallbacks ship in the build.

2. **Constraint: five weeks, one principal.** New engineering surface must stay near zero. *Therefore:* the game reuses the shipped combat stack wholesale; the only new gameplay systems are match/respawn flow, one stamina attribute, and bot state machines — content (arena, tuning, QA) is delegated to the dev-time crew as reviewable data.

3. **Constraint: small-PvP population risk.** An indie PvP game with an empty queue is dead at week two. *Therefore:* bots fill every empty slot in every mode at launch, PvE is the same game rather than a second one, and online is invite-first listen server.

4. **Constraint: API latency (1–4 s typical).** *Therefore:* zero model calls in the simulation path; 3-second timeout on flavor calls; the factual kill feed never waits.

4.4 Scope (shipped list) and Schedule

Shipped scope: FFA Deathmatch (2–4) and 2v2 TDM; 1 arena; 2 loadout archetypes (blade, spellblade) with 1 swappable magic slot each; stamina/ winded system; bots at 3 tiers filling any slot; 8-minute matches with sudden-death tiebreak; CommonUI front end (menu, lobby, HUD, scoreboard, kill feed); Caster Agent with canned fallback; MetaSounds combat audio; Steam demo build. **Nothing else.**

Explicitly cut: second arena, third archetype, ranked, quickmatch, objective modes, dedicated servers, cosmetics, achievements, 3v3+.

Week	Deliverable (each week ends runnable)	Gate
1	Arena blockout (crew); match timer + kill scoring + respawn; stamina + winded; FFA vs dumb bots, local	Committed exchanges feel good; ganks don't feel cheap
2	Steam listen-server host/invite; archetype 2; bot tiers 1–2; TDM scoring	Two humans + two bots, online, full match
3	Telemetry + Caster (canned first, API second); bot tier 3; swap-while-dead; MetaSounds cues	Full loop with audio and commentary

Week	Deliverable (each week ends runnable)	Gate
4	CommonUI front end; Balance Analyst pass on nightly QA soaks; spawn tuning	A stranger can navigate menu → match → rematch
5	Steam depot + demo build; overnight soaks; capstone presentation	Shipped

First cut if behind: Team Deathmatch — FFA alone carries the demo.

4.5 Success Criteria

- A stranger installs the Steam demo, fights bots inside 60 seconds, and hosts a friend without instructions.
- Playtesters run it back unprompted at match end.
- Every arena decision, bot tier, and balance change in the build traces to a named agent output that was human-reviewed.

Appendix A — First-Draft Combat Tuning

Parameter	Value	Note
Health	100	All archetypes
Stamina pool	100	All archetypes
Stamina regen	20/s after 1.0 s delay	Delay resets on any spend
Winded threshold	0 → locked until 30%	≈ 2.5 s helpless at full regen
Light attack	12 dmg, 15 stamina, 0.5 s	Chains up to 3 (12/12/16 dmg)
Heavy attack	28 dmg, 30 stamina, 1.1 s windup	Breaks block: 0.8 s guard-break stagger
Block (hold)	Negates lights, 10 stamina/hit	Movement –20% while holding
Parry window	0.25 s	Success: attacker staggered 1.5 s
Dodge	20 stamina, 0.4 s i-frames	0.3 s recovery, punishable
Firebolt	22 dmg projectile, 12 s CD	~18 m/s; dodgeable at range
Shockwave	18 dmg AoE, r = 4 m, 12 s CD	The anti-gank button
Match length	8:00	Sudden death on tie
Respawn	5 s	Farthest-scored spawn point
Fighter counts	FFA 2–4; TDM 2v2	Bots fill all empty slots

Appendix B — Bot Tier Design Targets

Tier	Fantasy	Aggression	Parry %	Reaction	Stamina discipline
1 — Brawler	Mashes, winds itself, punishable	High	5%	400 ms	Poor (spends to zero)
2 — Duelist	Spaces, blocks, occasionally parries	Medium	20%	300 ms	Fair (keeps 25% reserve)
3 — Warden	Baits, holds reserves, punishes greed	Adaptive	40%	220 ms	Strict (keeps 40% reserve)

All tiers use the same abilities, costs, and rules as human players — bots activate abilities through the same input path, so QA soak tests exercise the real combat code.

Appendix C — Telemetry Schema (per fighter, per match)

`FOS_MatchTelemetry`: kills, deaths, damage dealt/taken, lights/heavies thrown and landed, blocks held, parries attempted/landed/suffered, dodges, times winded, magic casts/hits, longest streak, time at leader position. Consumed by: Caster Agent (coach lines), Balance Analyst (win-rate bounds), Combat QA (regression baselines).